PRD: Developer Dashboard Web App

Document Owner: Navneet Shukla | Last Updated: 4-Jun-2026

Jira Epic	Link epic here
Delivery Date	TBD — to be set after PRD sign-off
Document Status	Draft
Product Lead	Navneet Shukla
Engineering Lead	TBD
Design Lead	TBD
QA Lead	TBD
Relevant Documents	Developer-Dashboard CLI repo, Claude Console export docs, Jira export docs

Table of Contents

1. Overview

Background

The engineering team at Cashflo already runs a CLI pipeline (Developer Dashboard) that merges Claude Console spend data with Jira deliverable and bug data to produce a monthly metrics CSV. The pipeline is functional and battle-tested with unit tests, but it lives entirely in a terminal. Results sit in a flat CSV file that must be shared manually, interpreted individually, and has no history, no visualisation, and no access for non-technical stakeholders.

Why are we doing this?

The CLI produces valuable data — AI efficiency scores, cost-per-deliverable, bug attribution, traffic-light flags — but no one outside the person running the script can see it. Engineering managers cannot check their team's status without asking someone to re-run and share a CSV. Developers have no visibility into their own numbers. Month-over-month trends are impossible to see. The data exists; the product does not.

How do we know this is a real problem worth solving?

- The CLI already exists and is being used, which validates that the underlying data is worth having.
- The output CSV has 15 columns per developer — this is not a trivial dataset to read raw.
- Traffic-light flags (green / yellow / red / gray) are computed but never surfaced visually anywhere.
- There is no current way for a developer to look up their own metrics without asking an admin.
- Month-over-month trend analysis is impossible today — previous output.csv files are overwritten each run.

How does this fit into company objectives?

- Supports the AI productivity measurement initiative — gives leadership visibility into ROI on Claude licences.
- Part of the engineering quality strategy — bug attribution data helps identify code quality patterns early.
- Contributes to engineering culture: transparent, data-driven performance visibility for developers themselves.

What problem is this solving?

A useful metrics pipeline produces output that no one can easily read, share, or act on. There is no web interface, no history, no visualisation, and no access control that lets different stakeholders see the data they need.

Who are we solving for?

- Engineering managers — need to see their team's status, spot red/yellow flags, and review trends.
- Individual developers — want to see their own metrics, understand their efficiency score, and track their progress over time.

What are the user benefits?

- Managers: a live dashboard replacing manual CSV sharing, with flag-based alerting and month-over-month trends.
- Developers: a personal profile page showing their own metrics, removing the opaque 'black box' feeling.
- Both: historical data that survives between runs, enabling trend analysis.

2. Goals

Turn the existing CLI pipeline into a web application that any team member can access in a browser, where the pipeline runs automatically and on-demand, and where results are stored, visualised, and browsable over time.

What success looks like

- An engineering manager opens a browser, sees the current month's team dashboard with traffic-light flags, and can drill into any developer's profile — without running a script or touching a CSV.
- A developer logs in and immediately sees their own metrics for the current and previous months.
- Historical data accumulates month-over-month so trends become visible over a quarter.
- The pipeline runs automatically on a schedule and a manual re-run button is available for on-demand refreshes.

3. Success Metrics

How do we know we have solved this problem?

Metric	Definition	Target
Adoption	% of engineering team who view the dashboard at least once per month	80% of team within 60 days of launch
Manual CSV sharing eliminated	Number of manual CSV exports sent via Slack/email after launch	Zero within 30 days of launch
Pipeline reliability	% of scheduled pipeline runs that complete successfully	≥ 95% success rate per month
Historical data depth	Number of months of trend data stored and browsable in the UI	≥ 3 months of data within 90 days of launch
Time-to-insight	Time from pipeline run completing to results visible in the UI	< 60 seconds

4. Assumptions & Dependencies

Assumptions

- The four input CSV files (Claude Console export, Jira deliverables, Jira bugs, developers map) are placed into a known server directory before or on the scheduled run date each month.
- The CSV file formats from Claude Console and Jira remain stable. Any schema changes upstream will require a loader update.
- All developers have email addresses that match between Claude Console and Jira exports (the existing developer map CSV bridges any mismatches).
- The server hosting the web app has filesystem access to the CSV input directory.
- Authentication is out of scope for v1 — the app is deployed on an internal network and treated as a trusted internal tool. All authenticated users see all data.
- The existing Python pipeline (merge.py and its modules) is not rewritten for v1 — the web app wraps and calls it.

Dependencies

- Existing CLI pipeline: merge.py, src/ modules, and tests/ — must remain passing throughout the web app build.
- Python runtime on the server (same environment as the CLI today).
- A web framework for the backend to expose pipeline triggers and serve data (to be decided — see Questions).
- A database or persistent store to retain historical output data per period (to be decided — see Questions).
- A frontend framework to render the dashboard, leaderboard, trend charts, and developer profiles.
- Server cron or equivalent scheduler for the automated monthly run.

5. Requirements

The requirements below are grouped by feature area. Each requirement maps to a user story with acceptance criteria, priority, and open notes.

5.1 Pipeline execution

#	Title	User story	Acceptance criteria	Priority	Notes
1	Scheduled auto-run	As an engineering manager, I want the pipeline to run automatically each month so that I don't have to remember to trigger it manually.	A cron job runs merge.py on the 1st of each month at 08:00. On success the results are persisted and the dashboard reflects the new data. On failure an error state is shown in the UI with the failure reason.	Must have	Schedule day/time TBD — see Questions. Cron config must be version-controlled.
2	Manual re-run	As an engineering manager, I want to trigger a pipeline run manually from the UI so that I can refresh data mid-month or after a CSV correction.	A 'Run pipeline' button is visible on the dashboard. Clicking it shows a period picker (YYYY-MM, defaulting to current month). On confirm, the pipeline runs, a progress indicator is shown, and results update on completion. If a run is already in progress the button is disabled.	Must have	Only one run at a time. Show last run time and status.
3	Run history log	As an engineering manager, I want to see a log of past pipeline runs so that I can tell when data was last refreshed and whether any runs failed.	A run log shows: timestamp, period, triggered by (scheduled / manual), status (success / failed), and duration. At minimum the last 10 runs are shown.	Must have	Persist run log in the database.

5.2 Team dashboard — leaderboard

#	Title	User story	Acceptance criteria	Priority	Notes
4	Team leaderboard table	As an engineering manager, I want to see all developers ranked in a table so that I can compare team performance at a glance.	A table shows all developers for the selected period. Columns: developer name, spend, lines of code, deliverables, story points, AI efficiency score, bugs attributed, bug rate, flag. Each flag is rendered as a colour badge (green / yellow / red / gray). Table is sortable by any column.	Must have	Default sort: flag severity then developer name.
5	Period selector	As an engineering manager, I want to switch between reporting periods so that I can view data for any past month.	A dropdown or date picker at the top of the dashboard lists all periods for which pipeline data has been stored. Selecting a period reloads the leaderboard for that month.	Must have	Default to the most recent period.
6	Flag filter	As an engineering manager, I want to filter the leaderboard to show only red and yellow developers so that I can focus on who needs attention.	A filter control (e.g. toggle or checkbox group) above the table allows filtering by flag value. Multiple flags can be selected. Selecting 'red' + 'yellow' hides green and gray rows.	Must have	Default: show all flags.

5.3 Developer profile page

#	Title	User story	Acceptance criteria	Priority	Notes
7	Developer profile	As a developer, I want to see my own metrics on a dedicated page so that I understand how I'm performing and what the numbers mean.	Clicking a developer's name in the leaderboard opens a profile page. The page shows: current period metrics (all columns from the leaderboard), a	Must have	Plain-language flag explanation is important — the score formula is opaque without it.

			plain-language explanation of their flag status, and a metric-by-metric breakdown.		
8	Month-over-month trend chart	As a developer, I want to see how my metrics have changed over the past months so that I can understand whether I'm improving.	The profile page includes a line chart showing ai_efficiency_score, spend_usd, and deliverables_count over all stored periods. The x-axis is YYYY-MM. Chart is interactive (hover for exact values). Only periods where data exists for that developer are plotted.	Must have	Use a charting library (to be decided). Minimum 3 periods to make trends meaningful.
9	Bug attribution detail	As a developer, I want to understand which bugs were attributed to me and why so that I can verify and learn from them.	The profile page lists each attributed bug with: bug issue key, epic, sprint, bug created date, and the story that triggered attribution. Links to Jira if Jira base URL is configured.	Nice to have	Requires Jira base URL config. Defer Jira linking if config is not available at launch.

5.4 Team trend view

#	Title	User story	Acceptance criteria	Priority	Notes
10	Team trend chart	As an engineering manager, I want to see how the team's aggregate metrics are trending month-over-month so that I can report on AI productivity improvement.	A 'Trends' view shows team-level aggregate charts: total spend, average AI efficiency score, total deliverables, and total bugs — one line per metric across all stored periods. Period range is selectable.	Must have	Aggregate = sum or average across all developers for that period.
11	Flag distribution over time	As an engineering manager, I want to see how the team's flag distribution has changed over time	A stacked bar chart shows the count of green / yellow / red / gray flags per period. Hovering a bar shows	Nice to have	Useful for QBR reporting. Defer if chart complexity is high.

		so that I can track whether quality is improving.	the developer names in each category.		

6. UX Mockups

Mockups and wireframes to be added here. The following screens are required before engineering kickoff:

- Dashboard — leaderboard table with period selector, flag filter, and 'Run pipeline' button.
- Dashboard — pipeline status bar (last run time, status badge, manual re-run button).
- Developer profile page — metric cards, flag explanation, month-over-month trend chart, bug attribution list.
- Team trends view — aggregate line charts and flag distribution stacked bar.
- Pipeline run log — table of past runs with status, timestamp, and duration.

Work with the Design Lead to produce wireframes in Figma. Link the main Figma file here once available.

7. Questions

Question	Outcome / Owner
What backend web framework should we use? Options: FastAPI (Python, minimal context-switch from existing code), Django, or Node/Express (requires bridging to the Python pipeline).	To decide with Engineering Lead before sprint 1. Recommendation: FastAPI to keep the stack homogeneous.
What database should store historical output data? Options: SQLite (simple, zero-infra, fine for a small team), PostgreSQL (more robust if the app scales), or flat JSON files per period.	To decide with Engineering Lead. SQLite is likely sufficient for v1 given team size.
What is the exact schedule for the automated monthly pipeline run? (Day of month, time, timezone.)	To decide with Product and Engineering. Recommendation: 1st of month, 08:00 IST, after Jira sprint close.
How are the input CSV files placed on the server? Is there a manual upload step or are they exported directly from Claude Console / Jira to a shared directory?	Must be defined before launch. If manual, document the exact file naming convention and directory path. If automated, scope the integration work separately.
Should the AI efficiency score formula be explained inline in the UI? (Currently: story_points / spend_usd, suppressed when coverage < 80%.) Developers unfamiliar with the formula may find their score confusing or unfair.	Recommendation: Yes — add a tooltip or info panel explaining the formula on both the leaderboard and the developer profile page.
Do we need a Jira base URL config so bug issue keys on the developer profile link directly to Jira tickets?	Nice-to-have for v1. Decide with Engineering Lead — low effort if Jira URL is stable.
What is the hosting target for the web app? Internal server, cloud VM, or a managed PaaS?	To decide with Engineering Lead before infrastructure setup.

8. Out of Scope

The following are explicitly not part of this product version and should not be built unless a separate scope decision is made:

- Authentication and login. The v1 app is an internal tool on a trusted network. Auth (SSO, OAuth, individual logins) is a future scope item.
- Role-based access control. All users see all developer data in v1.
- Direct API integrations with Claude Console or Jira. The app continues to rely on manually placed CSV files for v1. Automated API ingestion is a future enhancement.
- Drag-and-drop or browser-based CSV upload. File ingestion happens at the server filesystem level, not through the web UI.
- Email or Slack alerting when a developer's flag turns red. The dashboard is pull-based in v1; push notifications are future scope.
- Rewriting or replacing the existing Python CLI pipeline. The web app wraps merge.py; it does not replace it.
- Mobile-optimised layout. The dashboard is a desktop-first internal tool.
- Multi-organisation or multi-team support. The app serves a single Cashflo engineering team in v1.